

Multi-Agent Bug Detection & Auto-PR System

Project Report — Updated Technical Explanation | Author: Harsh Ganatra | 2026

Table of Contents

- 1. System Overview
- 2. Architecture & Pipeline
- 3. Phase 1 — Parallel Analysis (Agents 1–4)
- 4. Phase 2 — Bug Investigation (Agent 5)
- 5. Phase 3 — Repair Planning (Agent 6)
- 6. Phase 4 — Fix Generation (Agent 7)
- 7. Phase 5 — Validation & PR (Agents 8–10)
- 8. Confidence Scoring
- 9. Knowledge Graph Deep-Dive
- 10. End-to-End Walkthrough Example

1. System Overview

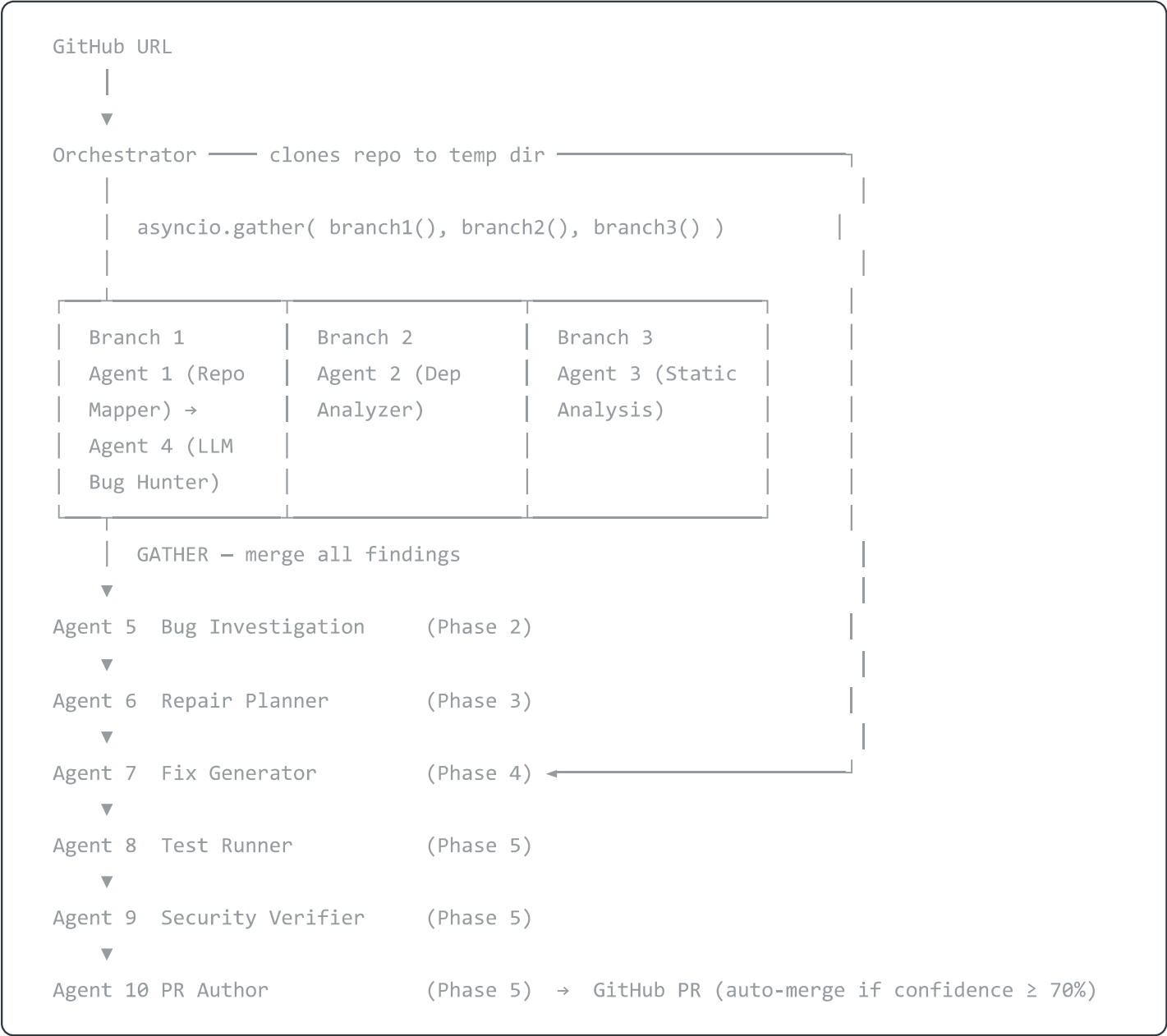
This system automates the full bug-detection-to-fix lifecycle for any GitHub repository. You give it a repo URL; it clones the code, runs 10 specialised agents across 5 phases, and — if it is confident enough — raises and *auto-merges* a pull request with the fix, all without human intervention.

Input	GitHub repository URL (public or private with token)
Output	One PR per buggy file (auto-merged when confidence ≥ 70 %)
LLMs used	Claude Haiku (cheap triage) + Claude Sonnet (accurate confirmation & fix)

Static tools	Bandit, ESLint, pip-audit, npm-audit, safety
Memory	ChromaDB vector store — stores past fixes for semantic reuse
Graph	NetworkX DiGraph — models every file, function, and import relationship

2. Architecture & Pipeline

The **Orchestrator** is the conductor. It is not one of the 10 agents; it manages them. It uses `asyncio.gather()` to run independent work in parallel and `await` to wait for results before the next phase begins.



3. Phase 1 — Parallel Analysis

Phase 1 runs **three concurrent branches** via a single `asyncio.gather()` call. The key insight is that Branch 1 is actually two agents chained sequentially — Agent 4 cannot start until Agent 1 has built the knowledge graph. Branches 2 and 3 are fully independent and need no graph, so all three branches start at the same instant.

BRANCH 1 — SEQUENTIAL CHAIN

Agent 1 (Repo Mapper)
↓ builds graph

Agent 4 (LLM Bug Hunter)
↓ uses graph

Agent 4 waits for Agent 1's graph before it can rank files by connectivity.

BRANCH 2 — INDEPENDENT

Agent 2
(Dependency Analyzer)

Reads requirements.txt / package.json. No graph needed.
Runs pip-audit, npm audit, safety.

BRANCH 3 — INDEPENDENT

Agent 3
(Static Analysis)

Runs Bandit + ESLint on source files. No graph needed. Pattern-based detection.

```
# orchestrator.py – the actual asyncio.gather call
async def _map_then_hunt(kg_ref):
    kg, findings1 = await agent1.run(repo_path)    # Agent 1 first
    kg_ref.append(kg)
    findings4 = await agent4.run(repo_path, kg)    # Agent 4 uses result
    return findings1 + findings4

findings_a, findings_b, findings_c = await asyncio.gather(
    _map_then_hunt(kg_holder),    # Branch 1 – chains internally
    agent2.run(repo_path),        # Branch 2 – independent
    agent3.run(repo_path),        # Branch 3 – independent
)
all_findings = findings_a + findings_b + findings_c
```

Agent 1 Repo Mapper

Agent 1's job is to build a **NetworkX directed graph** that models the entire codebase as a graph of nodes (files, functions, classes) and edges (imports, calls, queries). This graph is used by every downstream agent for impact analysis.

How it works — 4 build passes

1. **Pass 1 — Structure:** walks every file, creates a node per file with metadata (size, language, last modified).
2. **Pass 2 — AST:** for each Python file, calls `ast.parse()` to extract all functions, classes, and `import` statements. Adds IMPORTS edges between files.
3. **Pass 3 — Framework:** detects Flask/Django/FastAPI routes, SQLAlchemy models, REST endpoints. Adds DEFINES_ROUTE, DEFINES_MODEL edges.
4. **Pass 4 — Dynamic:** traces function call patterns across files. Adds CALLS and QUERIES_DB edges.

EXAMPLE — FLASK TODO APP

Given this repo structure:

```
todo_app/  
  app.py      ← imports models.py, imports db.py  
  models.py   ← imports db.py  
  db.py       ← standalone  
  routes/  
    tasks.py  ← imports models.py, imports app.py
```

Agent 1 builds this graph:

```
Nodes: app.py, models.py, db.py, routes/tasks.py  
Edges (IMPORTS):  
  app.py      → models.py  
  app.py      → db.py  
  models.py   → db.py  
  tasks.py    → models.py  
  tasks.py    → app.py
```

Also finds syntax errors during `ast.parse()` — e.g. a missing colon in a function definition becomes a `SyntaxError` Finding immediately.

What it returns

A tuple `(KnowledgeGraph, list[Finding])`. The graph goes to Agent 4; the findings (syntax errors, circular imports, compile errors) join the main finding pool.

Agent 2 Dependency Analyzer

Scans `requirements.txt` and `package.json` for known CVEs using three tools run concurrently: **pip-audit**, **safety**, and **npm audit**.

EXAMPLE FINDING

```
# requirements.txt contains:
Flask==1.0.2
requests==2.18.4

# pip-audit finds:
CVE-2023-30861 in Flask 1.0.2 — Cookie header bypass
  Fix: upgrade to Flask >= 2.3.2

PYSEC-2022-42969 in requests 2.18.4 — Header injection
  Fix: upgrade to requests >= 2.28.0
```

Returns a `list[Finding]`. Each finding has `severity`, `cve_id`, `affected_package`, `fix_version`.

Agent 3 Static Analysis

Runs **Bandit** (Python security linter) and **ESLint** (JavaScript/TypeScript linter) concurrently. Catches known bad patterns — not logic bugs, just things that match a rule.

EXAMPLE FINDINGS FROM BANDIT

```
# In db.py:
password = "admin123"           # B105: hardcoded password
```

```
query = "SELECT * FROM users WHERE id=" + user_id # B608: SQL injection pattern

# In utils.py:
data = pickle.loads(user_input) # B301: unsafe pickle
eval(user_expression)           # B307: use of eval()
```

Bandit assigns each finding a `severity` (LOW/MEDIUM/HIGH) and `confidence` (LOW/MEDIUM/HIGH). Agent 3 filters to MEDIUM+ severity.

Returns `list[Finding]`. These are pattern-based — Agent 3 cannot tell you *why* the logic is wrong, only that it matches a known dangerous pattern.

Agent 4 LLM Bug Hunter

This is the most sophisticated Phase 1 agent. It uses the knowledge graph from Agent 1 to *rank files by importance*, then runs a **two-stage LLM triage**: cheap Haiku first, expensive Sonnet only for confirmed candidates.

Stage A — Haiku Triage (one file at a time)

`_select_files()` sorts files by *graph degree* (number of in + out edges) — more connected = more impactful if buggy = scan first. Then for each file, Agent 4 sends:

- The file's full source code
- A small graph context snippet: which files import this file, which it imports
- A prompt asking for a JSON list of *candidate* bug locations

Haiku is cheap (~\$0.00025/1K tokens), so scanning every file this way is affordable. Haiku returns a list of candidates — it is allowed to have false positives.

Stage B — Sonnet Confirmation (with related files)

For each Haiku candidate, Agent 4 builds a richer context:

- The primary file (full source)
- All related files from the graph (files that import it or that it imports, up to 3 hops)
- The candidate locations from Stage A

Sonnet (~\$0.003/1K tokens) confirms or rejects each candidate. A confirmed finding includes

`root_cause`, `evidence` (exact lines), and a `suggested_fix`.

EXAMPLE — SQL INJECTION BUG THAT BANDIT MISSED

```
# tasks.py – Haiku triage prompt (simplified):
# "Here is tasks.py. List any bug candidates as JSON."

# Haiku output:
[{"line": 34, "issue": "possible SQL injection in get_task()"}]

# Sonnet confirmation – sees tasks.py + models.py + db.py together:
# Confirms: tasks.py line 34 builds a raw SQL string with user input.
# models.py line 12 defines the Task model, db.py line 5 creates the connection.
# Root cause: user_id from request is concatenated directly, no parameterisation.
# Suggested fix: use parameterised query:
#     cursor.execute("SELECT * FROM tasks WHERE id = ?", (user_id,))
```

Returns `list[Finding]` with richer context than Agents 2 and 3 — includes logic-level bugs, not just pattern matches or CVEs.

4. Phase 2 — Bug Investigation (Agent 5)

Agent 5 Bug Investigator

Phase 1 gives us a raw list of findings from all three branches — possibly 30–50 items. Agent 5's job is to **enrich each finding** with impact analysis: what other files break if this bug is present? How far does the damage spread?

For each finding, Agent 5 calls three knowledge-graph methods:

1. `kg.analyze_change(file, hops=2)`

Walks the graph *backwards* — finds all files that **consume** the buggy file. If `tasks.py` is buggy, `analyze_change` finds every file that imports `tasks.py`, then every file that imports those, up to 2 hops.

```
# tasks.py is buggy
kg.analyze_change("routes/tasks.py", hops=2)

# Graph edges (IMPORTS direction):
# app.py → routes/tasks.py (app.py imports tasks.py)
# test_app.py → app.py (test_app.py imports app.py)

# analyze_change walks BACKWARDS (reverse edges):
# routes/tasks.py ← app.py ← test_app.py
# Returns: ["app.py", "test_app.py"] ← these will break too
```

2. `kg.blast_radius(file, hops=2)`

Calls `analyze_change()` and then adds functions, API endpoints, and DB models that are defined in or used by the affected files. This gives a complete picture of *everything* that is at risk.

```
kg.blast_radius("routes/tasks.py")
# Returns:
{
  "affected_files": ["app.py", "test_app.py"],
  "affected_functions": ["create_task()", "get_task()", "delete_task()"],
  "affected_apis": ["/tasks", "/tasks/"],
  "affected_db_models": ["Task"]
}
```

3. `kg.trace_data_flow(file, line)`

For the specific buggy line, traces *where the data comes from* (upstream callers) and *where it goes* (downstream callees). This tells us: who feeds data into the vulnerable function, and who depends on its output.

```
# Bug at tasks.py line 34: raw SQL with user_id
kg.trace_data_flow("routes/tasks.py", 34)
# Returns:
{
  "upstream": ["app.py:handle_request() → tasks.py:get_task()"],
```



```
"downstream": ["tasks.py:get_task() → db.py:execute_query()"]
}
```

So: user input enters via handle_request, passes to get_task, hits execute_query
The entire chain is tainted.

ChromaDB — Similar Past Fixes

Agent 5 also queries ChromaDB with the finding's description. ChromaDB does a *semantic similarity search* over all past fixes stored as embeddings. If a similar bug was fixed before, Agent 5 attaches the past fix as a hint for the repair planner.

```
# Query: "SQL injection via string concatenation in query builder"
# ChromaDB returns:
{
  "similar_fix": "Use parameterised queries: cursor.execute(sql, (param,))",
  "similarity_score": 0.91,
  "source_commit": "fix/sql-injection-auth-2024-03-15"
}
```

Finally, Agent 5 calls the LLM for root-cause analysis **only if Agent 4 did not already provide one** — this avoids paying twice for the same LLM call.

Returns the same `list[Finding]` but each item now has: `blast_radius`, `data_flow`, `similar_past_fix`, `root_cause`.

5. Phase 3 — Repair Planning (Agent 6)

Agent 6 Repair Planner

Agent 6 takes the enriched findings list and produces an **ordered repair plan**. The key problems it solves:

- Multiple findings may be in the same file — combine them into one PR instead of many.
- Some files depend on other files — fix dependencies *before* dependents.
- Security bugs should be fixed before logic bugs.

Step 1 — Group by file

```
# Findings from Phase 2:
# Finding A: tasks.py — SQL injection (HIGH severity)
# Finding B: tasks.py — missing input validation (MEDIUM)
# Finding C: models.py — insecure default value (LOW)
# Finding D: db.py — outdated library (CVE)

# After grouping:
# tasks.py   → [Finding A, Finding B]
# models.py  → [Finding C]
# db.py      → [Finding D]
```

Step 2 — Build file dependency graph & topological sort

```
# File import graph (from knowledge graph):
# tasks.py imports models.py
# tasks.py imports db.py
# models.py imports db.py

# Topological sort (dependencies first):
# 1. db.py      (no dependencies — fix first)
# 2. models.py  (depends on db.py)
# 3. tasks.py   (depends on both)

# Security files (CVE findings) are promoted to top of their tier.
# Final order: db.py → models.py → tasks.py
```

Step 3 — Assign initial confidence

Each `RepairPlanItem` starts with a pre-score:

- Base: **0.50**
- +0.15 if ChromaDB returned a similar past fix (cache hit)
- = 0.65 with cache hit, 0.50 without

This is a starting point; Agents 8 and 9 adjust it upward or downward after testing.

Returns an ordered `list[RepairPlanItem]`. Each item holds: `file`, `findings`, `fix_strategy` (combined prompt for all findings in that file), `confidence_pre_score`, `fix_order`.

6. Phase 4 — Fix Generation (Agent 7)

Agent 7 Fix Generator

For each `RepairPlanItem` from Agent 6, Agent 7 calls Claude Sonnet with the full file source, the combined fix strategy, related files context, and any similar past fix from ChromaDB. Sonnet returns a patch in **SEARCH/REPLACE format**.

SEARCH/REPLACE patch format

Rather than generating an entire new file (risky — could change unrelated lines), the LLM produces anchored edit blocks. Each block specifies an exact string to find and the replacement. The system applies them sequentially.

```
# LLM output for tasks.py SQL injection fix:

<<<<<<< SEARCH
def get_task(user_id):
    query = "SELECT * FROM tasks WHERE id=" + user_id
    cursor.execute(query)
=====
def get_task(user_id):
    query = "SELECT * FROM tasks WHERE id = ?"
    cursor.execute(query, (user_id,))
>>>>>>> REPLACE
```

The SEARCH string must match *exactly* (including whitespace). If it doesn't match, the patch is rejected and Agent 7 retries with a fresh context. Patches can span multiple files — one repair plan item can produce edits across `tasks.py` and `models.py` if both need changes.

ChromaDB write-back

After generating the fix, Agent 7 stores the (bug description → fix) pair in ChromaDB. Future runs on similar codebases will find this as a "cache hit" and get a head-start on confidence scoring.

Returns a `list[RepairResult]` — each with the patched file content, the diff, and the patch as applied.

7. Phase 5 — Validation & PR (Agents 8–10)

Phase 5 has a **retry loop**: if Agent 8 (tests) or Agent 9 (security) reject the fix, control returns to Agent 7 for a revised patch. Up to 3 iterations per file.

Agent 8 Test Runner

Applies the patch to the real files in the cloned repo, then runs the project's test suite (`pytest`, `npm test`, `go test`, etc. — auto-detected). Captures exit code, number of passing/failing tests, and stderr.

```
# After patching tasks.py:
pytest tests/ -x --tb=short
# Result: 14 passed, 0 failed → test_score = 1.0 (+0.40 to confidence)
# Result: 12 passed, 2 failed → test_score = 0.86 (+0.34)
# Result: 0 passed (crash) → test_score = 0.0 → trigger retry
```

Test score contributes **40%** to the final confidence score — it is the largest single signal. If tests fail catastrophically, Agent 8 returns a failure result and the orchestrator sends the patch back to Agent 7 with the test output as feedback.

Agent 9 Security Verifier

Re-runs Bandit and the dependency scanner on the *patched* file. Checks:

- Did the patch introduce any *new* security issues?

- Did it actually fix the reported issue (original finding no longer triggered)?
- Is the fix semantically sound (LLM-based check for logic correctness)?

```
# Before patch: B608 SQL injection found at line 34
# After patch:  B608 no longer triggered ✓
# New issues:   none ✓
# Semantic check (Haiku): "Fix looks correct – uses parameterised query" ✓
# security_score = 1.0  (+0.25 to confidence)
```

Security contributes **25%** of final confidence. If a new HIGH severity issue appears, Agent 9 triggers a retry.

Agent 10 PR Author

Agent 10 creates the GitHub PR and, if confidence is high enough, auto-merges it.

```
# Decision logic:
requires_approval = (confidence < 0.70) or (file in critical_path_files)

if requires_approval:
    # Create a DRAFT PR – human must review before merge
    github.create_pull_request(..., draft=True)
else:
    # Create a normal PR then immediately merge it
    pr_url = github.create_pull_request(...)
    merged = github.merge_pull_request(repo_owner, repo_name, pr_url,
                                       commit_message=title)
```

The `merge_pull_request()` method polls GitHub's `mergeable` flag up to 5 times (2 s apart) because GitHub computes merge-ability asynchronously. Once mergeable, it squash-merges.

PR description generated by Agent 10:

```
## Bug Fix: SQL Injection in routes/tasks.py

**Root cause:** User-supplied `user_id` was concatenated directly into a
SQL string at line 34, allowing an attacker to escape the query.
```

```
**Fix applied:** Replaced string concatenation with a parameterised query
using `cursor.execute(sql, (param,))`.

**Blast radius:** app.py, test_app.py, Task model, /tasks API endpoints.

**Confidence:** 87% (tests: 14/14 pass, security scan clean, cache hit)

**Auto-merged:** Yes (confidence ≥ 70%, not on critical path)
```

8. Confidence Scoring — 5-Signal Model

Every repair goes through five independent signals. The final confidence score decides whether to auto-merge or draft-PR.

Signal	Weight	How it's measured	Score range
Tests	40%	pass_count / total_count from test runner	0.0 – 1.0
Security	25%	Bandit re-scan: no new issues + original issue gone	0.0 – 1.0
Cache hit	15%	ChromaDB similarity ≥ 0.85 → score 1.0, else 0	0.0 or 1.0
AST validity	10%	<code>ast.parse()</code> succeeds on patched file	0.0 or 1.0
Fix order	10%	Penalties if security bugs not fixed first; bonus if topological order respected	0.5 – 1.0

```
confidence = (
    0.40 * test_score +
    0.25 * security_score +
    0.15 * cache_score +
    0.10 * ast_score +
    0.10 * order_score
)
```

```
# Example – all signals green:
# 0.40*1.0 + 0.25*1.0 + 0.15*1.0 + 0.10*1.0 + 0.10*1.0 = 1.00
# → auto-merge

# Example – tests pass but no cache hit, no security scan:
# 0.40*1.0 + 0.25*0.0 + 0.15*0.0 + 0.10*1.0 + 0.10*0.8 = 0.58
# → draft PR (below 0.70 threshold)
```

9. Knowledge Graph Deep-Dive

The knowledge graph (`backend/knowledge_graph.py`) is a `networkx.DiGraph` where **direction matters**. Edge direction = "A imports B" means edge goes A → B.

Node types & edge types

Node type	Examples
FILE	routes/tasks.py, models.py, db.py
FUNCTION	get_task(), create_task(), execute_query()
CLASS	Task, User, DatabaseManager
ROUTE	GET /tasks, POST /tasks/<id>

Edge type	Meaning	Direction
IMPORTS	File A imports file B	A → B
CALLS	Function A calls function B	A → B
DEFINES	File A defines function/class B	A → B
QUERIES_DB	Function A queries model B	A → B
DEFINES_ROUTE	File A defines API route B	A → B

Why direction matters for impact analysis

```
# Graph: app.py → tasks.py → db.py
#       (app imports tasks; tasks imports db)

# If db.py is buggy — who is affected?
# We need files that CONSUME db.py — walk BACKWARDS (reverse edges):
# db.py ← tasks.py ← app.py
# Both tasks.py and app.py will break if db.py has a bug.
# This is analyze_change() — it reverses the import direction.

# If we walked FORWARDS from db.py, we'd get nothing (db.py imports no one).
# Forward traversal answers "what does db.py depend on?" — wrong question.
```

10. End-to-End Walkthrough — Flask Todo App

Putting it all together with a concrete example: a Flask todo app with a SQL injection bug.

INPUT

```
Repo URL: https://github.com/example/flask-todo-app
Files: app.py, routes/tasks.py, models.py, db.py, tests/test_tasks.py
```

Phase 1 — Parallel Analysis

```
t=0s  Orchestrator clones repo → /tmp/flask-todo-app/

t=1s  asyncio.gather() fires three branches simultaneously:

Branch 1: Agent 1 starts building knowledge graph
  → ast.parse() on all .py files
  → finds: app.py imports tasks.py, tasks.py imports db.py
  → graph built at t=8s
  → Agent 4 starts (has graph now)
    → Stage A: Haiku scans tasks.py (highest graph degree)
      → finds candidate: line 34, possible SQL injection
    → Stage B: Sonnet confirms with tasks.py + db.py + models.py context
      → Finding: SQL injection at tasks.py:34, severity=HIGH
```


Branch 2: Agent 2 (runs concurrently from t=1s)

→ pip-audit scans requirements.txt

→ Finding: Flask 1.0.2 CVE-2023-30861, fix=Flask>=2.3.2

Branch 3: Agent 3 (runs concurrently from t=1s)

→ Bandit scans all .py files

→ Finding: tasks.py:34 B608 SQL injection pattern (confirms Agent 4)

→ Finding: db.py:5 B105 hardcoded password

t=22s GATHER — all branches complete

all_findings = [SQL injection, Flask CVE, SQL pattern, hardcoded password]

Phase 2 — Bug Investigation

Agent 5 enriches each finding:

For SQL injection in tasks.py:34:

analyze_change("tasks.py") → ["app.py"] (app.py imports tasks.py)

blast_radius("tasks.py") → {

affected_files: ["app.py", "tests/test_tasks.py"],

affected_apis: ["GET /tasks", "GET /tasks/"],

affected_functions: ["get_task()", "list_tasks()"]

}

trace_data_flow("tasks.py", 34) → {

upstream: ["app.py:handle_request() → tasks.py:get_task()"],

downstream: ["tasks.py:get_task() → db.py:cursor.execute()"]

}

ChromaDB query: "SQL injection string concatenation"

→ similar past fix found (score 0.91): "use parameterised queries"

root_cause: already provided by Agent 4 — skip LLM call ✓

Phase 3 — Repair Planning

Agent 6 groups findings:

tasks.py → [SQL injection (HIGH), SQL pattern (MEDIUM)] → combined fix

db.py → [hardcoded password (MEDIUM)]

requirements.txt → [Flask CVE (HIGH)]

Dependency order: requirements.txt → db.py → tasks.py

(tasks.py imports db.py, so db.py fixed first)

Flask CVE (security) promoted ahead of db.py password.

Final order: requirements.txt → db.py → tasks.py

Confidence pre-scores:

```
requirements.txt: 0.50 + 0.15 (cache hit) = 0.65
db.py:            0.50 + 0.00              = 0.50
tasks.py:         0.50 + 0.15 (cache hit) = 0.65
```

Phase 4 — Fix Generation

Agent 7 generates SEARCH/REPLACE patches:

Patch 1 – requirements.txt:

```
Flask==1.0.2 → Flask==2.3.2
```

Patch 2 – db.py:

```
password = "admin123" → password = os.environ.get("DB_PASSWORD", "")
```

Patch 3 – tasks.py:

```
query = "SELECT * FROM tasks WHERE id=" + user_id
cursor.execute(query)
→
query = "SELECT * FROM tasks WHERE id = ?"
cursor.execute(query, (user_id,))
```

Phase 5 — Validation & PR

Agent 8 (Test Runner):

```
Apply all patches → run pytest tests/
Result: 14 passed, 0 failed
test_score = 1.0
```

Agent 9 (Security Verifier):

```
Re-run Bandit on patched files:
→ B608 SQL injection: gone ✓
→ B105 hardcoded password: gone ✓
→ New issues: none ✓
security_score = 1.0
```

Final confidence = 0.40*1.0 + 0.25*1.0 + 0.15*1.0 + 0.10*1.0 + 0.10*1.0

= 1.00 → AUTO-MERGE ✓

Agent 10 (PR Author):
Branch: fix/sql-injection-tasks-py-20260818
PR title: "fix: SQL injection and hardcoded credentials (3 files)"
PR created → merge_pull_request() polls mergeable flag → squash-merges
PR auto-merged ✓

Summary Table — All 10 Agents

#	Agent	Phase	Input	Output	Tools Used
1	Repo Mapper	1	repo path	KnowledgeGraph + syntax findings	ast.parse(), NetworkX
2	Dependency Analyzer	1	requirements.txt / package.json	CVE findings	pip-audit, safety, npm audit
3	Static Analysis	1	source files	pattern-based findings	Bandit, ESLint
4	LLM Bug Hunter	1	source files + KG	logic/context bug findings	Haiku (triage) + Sonnet (confirm)
5	Bug Investigator	2	all findings + KG	enriched findings with blast radius	NetworkX graph traversal, ChromaDB
6	Repair Planner	3	enriched findings	ordered repair plan	topological sort, file dep graph
7	Fix Generator	4	repair plan + source	SEARCH/REPLACE patches	Claude Sonnet, ChromaDB write
8	Test Runner	5	patched files	test_score, pass/fail counts	pytest / npm test / go test
9	Security Verifier	5	patched files	security_score, new issue check	Bandit re-scan, Haiku semantic check
10	PR Author	5	patches + confidence	GitHub PR (auto-merged if ≥ 70%)	PyGitHub, squash merge

